Worksheet 2: Version Control

Kosmas Hench
2025-05-26

Contents

1 git local 1
1.1 basic git usage 1
1.2 git time travels 3
1.3 Branching Out 4
2 Connecting to the Outside World 6

This Session's practical part is going to introduce the most relevant `git` *functionalities that you might need to effectively use version control for your projects/analyses. Admittedly, the examples are toys, but I hope they serve to demystify the process, and allow focussing on the actual workings of* `git`*. Also, in this worksheet the plain* `git` *commands are given in a form that can be run directly from the terminal/command line. This might appear daunting at first, but it is the most precise way to describe the individual steps. Rest assured that more user-friendly ways of managing* `git` *are available, particularly when using an IDE such as* `VS code` *or* `Rstudio`*. If you feel that you understand the intention behind the individual commands, there is nothing keeping you from using these options (we'll talk about this – if I forget, please remind me...).*

1. GIT LOCAL

The core job of version control is to keep track of the changes of your projects while they are being developed on your computer. For this `git` is keeping a paper trail of all the edits to files within a project. The basics and main work-horses for this task are two commands: "`git add`" and "`git commit`" – these two commands are probably about 98% of what is needed to use `git` for reproducible coding. The rest is fluff to deepen the understanding of `git`, to give a little context and to provide some starting point if things appear to be going haywire.

1.1 basic git usage

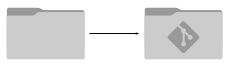
Prior to using `git` on a computer for the first time, you will be need to set your user name and email address. Use the ones from your github account for that.

```
git config --global user.email "you@example.com"
git config --global user.name "Your Name"
```


We start out by creating the brand-new, empty folder (our project folder to be).

```
# --- setting up a git repository ---
# create empty new folder - no git yet
mkdir project_folder
cd project_folder
ls -a
```

```
. ..
```


Using the comand `git init`, we can turn any regular folder into a `git` repoistory. This will create a hidden folder called `.git` inside the repository. Inside this hidded folder, `git` manages all the book-keeping and stores the information needed for the version control of the repository content.

```
# turn normal folder into git repo
git init
```

```
Initialized empty Git repository in <path_to>/project_folder/.git/
```

```
ls -a
.  ..  .git
```

Once we have a functional `git` repository, we can start using other `git` sub-comands. Here, we use `git status` to confirm that the creation of the repository was sucessfull.

```
# checking git status (confirming folder is a repo)
git status
On branch main
No commits yet
nothing to commit (create/copy files and use "git add" to track)
```

As soon as we have turned the folder into a repository, `git` will notice if the files therein change. In the example, we create a new text file (`A.txt`), that will then show up in the output of `git status`.

```
# --- start changing folder conten ---
# creating new file, checking git monitoring
echo "AAA" > A.txt
git status
On branch main
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        A.txt
nothing added to commit but untracked files present (use "git add" to track)
```

However, if we want to tell `git` to actually log changes to the files within the repository, we need to explicitly select the files that should be logged / saved. Here, we stage file `A.txt` for the next `comit` (save to the status of the `git` repository).

```
# staging file "A.txt" for commt (select it to log changes)
git add A.txt
git status
On branch main
No commits yet
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   A.txt
```

In a repository with staged files, we can then create a `git commit`. This will log the current state of the repository (of these files) and will allow us to go back to this state of the folder. To not lose track of many saved states, we need to label the `commit` with a tag.

```
# commit the changes to the staged files (actually log the changes)
git commit -m "added A"
[main (root-commit) 922a798] added A
1 file changed, 1 insertion(+)
create mode 100644 A.txt
```

```
# commit the changes to the staged files (actually log the changes)
git commit -m "added A"
```

From now on, using version control is pretty much *"rinse and repeat"*: We add/edit files, stage them and commit changes.

```
# add more edits to existing file and create additional new file
echo "aaa" >> A.txt
echo "BBB" > B.txt
# stage and commit ALL files with changes
git add .
git commit -m "added B"
```

If we want to include files in the folder that should be excluded from the version control, we can list them in a file called `.gitignore`.

```
# anticipate a file that should not be tracked in version control ("C.txt")
echo "C.txt" > .gitignore
# edit existing file and create "private" file
echo "bbb" >> B.txt
echo "CCC" > C.txt
# the ignored file should not show up in the git records
git status
```

Those files will not be tracked by `git` and thus `C.txt` is not appearing in `git` status:

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   B.txt
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
no changes added to commit (use "git add" and/or "git commit -a")
```

The files listed in `.gitignore` will also not be staged with the catchall `git add .` and thus they will be omitted from future commits.

```
git add .
git commit -m "expanded B"
```

1.2 git time travels

The main point of version control is that it enables us to restore previous states of the `git` repository. Specifically, we can reset the folder content to the state it was at any given `git commit` that we saved in the past. We can get an overview of all past commits using `git log`:


```
# --- start time travels ----
# overview of past commits
git log --pretty=oneline
```

```
1722479dd2e3ca52c5ae0fcb73dda7eb511dd6bf (HEAD → main) expanded B
ee82a3627e8a275f461f48e72ef2c7788d018714 added B
922a7980578ee844516e76abd278be3311c1dd49 added A
```

To see the effect of this restoration we first check the current state of the repository (before going back to a previous commit). At this point we see all three files (`A.txt`, `B.txt` and `C.txt`).

```
# list all files in project folder
tree
```

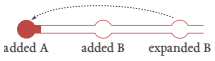
```
├── A.txt
├── B.txt
└── C.txt
1 directory, 3 files
```

We also see that the file `A.txt` contains the "full text" that we added over several commits.

```
# check content of A.txt
cat A.txt
```

```
AAA
aaa
```

Now, we use `git checkout` to revert to a previous state (in this case the very first commit "added A"). Note how the file `B.txt` disappears, yet `C.txt` remains unaffected. This is because we requested `C.txt` to be ignored by `git`.



```
# restore the project state at the time point of the first commit
git checkout ':/added A'
tree
```

```
├── A.txt
└── C.txt
1 directory, 2 files
```

Also, when looking at the content of `A.txt`, we see that the edits to the file that we included in the second commit are not in there yet:

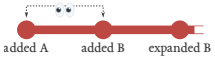
```
# re-check content of A.txt
cat A.txt
```

```
AAA
```

We can go back to the most recent state of the repository by checking out to the name of the respective `branch` (here "main"):


```
# return to the most recent project state
git checkout main
```

If we just want to check what exactly has changed between two commits (without necessarily restoring all files), we can use `git diff`. This will highlight the edited lines of all the files that have been added or changed between the two commits.



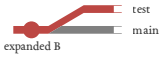

```
# display the tracked changes between the 1.& 2. commit
git diff ':/added A' ':/added B'
```

```
diff --git a/A.txt b/A.txt
index 43d5a8e..7393285 100644
--- a/A.txt
+++ b/A.txt
@@ -1 +1,2 @@
 AAA
+aaa
diff --git a/B.txt b/B.txt
new file mode 100644
index 0000000..ba62923
--- /dev/null
+++ b/B.txt
@@ -0,0 +1 @@
+BBB
```

1.3 Branching Out

To create a new branch (eg for experimental drafts), we can use the command `git branch`.

```
# --- using multiple branches ----
# create a new branch called "test"
git branch test
```



We can re-use `git checkout` also to activate and toggle between different branches within a `git` repository. Initially, switching to a newly created branch will look like there has been no effect, as the branch continues exactly from wherever it was created.

```
# switch to the new test branch
git checkout test
```

```
Switched to branch test
```

However, `git status` will now indicate that we are no longer on the "main" branch, but on "test" instead.

```
# just checking
git status
```

```
On branch test
nothing to commit, working tree clean
```

Now, on the test branch we can edit files and stage and commit them just like we did before.

```
# edtit file "A.txt" on test branch, stage and commit
echo "111" >> A.txt
git add .
git commit -m "expanded A"
```

Yet, when we switch back to the "main" branch, these edits will not appear in the files.

```
# switch back to the main branch
git checkout main
cat A.txt
```

```
AAA
aaa
```

To actually integrate the changes that we did in the test branch also into "main", we can use the command "git merge".

```
# integrate the changes from the test branch
git merge test
```

```
Updating 1722479..91a5d81
Fast-forward
 A.txt : 1 +
 1 file changed, 1 insertion(+)
```

After that, the edits will also show up in the files of "main".

```
cat A.txt
```

```
AAA
aaa
111
```

When juggling with different branches (or also when working on a project from multiple computers, eg. through github), it is possible for conflicts to arise. This happens when `git` has to merge two new versions of a single file that are contradicting each other. In the following, we are going to provoke such an example and resolve the resulting merge conflict.

For this, we first add another line to the file `B.txt` on the main branch. As usual, we stage and commit these edits.

```
# now, do some additions to "B.txt" on the main branch, stage and commit
echo "bbb" >> B.txt
git add .
git commit -m "more B edits"
```

Then we switch back to the test branch (that is unaffected by the last change to "main"). We also edit and commit `B.txt` on "test", but differently than we did on "main".

```
# switch back to test branch
git checkout test
# also modify "B.txt" on the test branch (and stage and commit)
echo "222" >> B.txt
git add .
git commit -m "test B edits"
```

Now, when going back to "main" and trying to integrate the changes we are presented with an error message because `git` does not know which of the two edits to `B.txt` is the desired one (or if it is to combine them – and if so, how).

```
# going back to the main branch and trying to integrate the edits from "test"
git checkout main
git merge test
```
```
Auto-merging B.txt
CONFLICT (content): Merge conflict in B.txt
Automatic merge failed; fix conflicts and then commit the result.
```

When looking at the offending file, we can see that `git` has highlighted the lines that caused the conflict and reports the line versions for both branches.

```
# checking the conflicting file
cat B.txt
```
```
BBB
bbb
<<<<<< HEAD
bbb
=======
222
>>>>>> test
```

To resolve this issue, we modify the highlighted file (use any editor of choice) to create the desired compromise of the two incompatible suggestions (in this case we keep both edits, with the line from the "main" branch first).

```
# the conflicting file after resolving the confilct
cat B.txt
```
```
BBB
bbb
bbb
222
```

Once the conflict has been handled, we once more stage and commit the curated file. After that we are back to normal, and we can proceed with the standard routine of *edit, stage and commit*.

```
# the resolved conflict needs to be staged and commited to complete the process
git add .
git commit -m "resolved conflict"
```
```
[main b2a285f] resolved conflict
```

2. Connecting to the Outside World

So far, we have been using `git` solely for the purpose of version control locally on a single computer (e.g. your laptop). Yet, a second main benefit of `git` is that it makes it easy to share code with others. We can connect a local `git` repository with github to make it publicly available, and from there it is easy to archive specific versions (a particular commit) to zenodo (thereby creating a DOI for the code and assuring the long-term availability of the actual analysis code of a study).

To connect our repository to a github url, we use the command `git remote add`. A `git remote` needs a label, so in this case we go with the default label "origin".

```
# --- connecting to a remote (using github) ---
# adding a url as a "remote" to setup a online address for synchronizing (github)
git remote add origin https://github.com/<usr_name>/project_folder.git
```

Once we have established a remote, we can synchronize local changes with `git push` (stating the target remote, as well as the branch that we want to synchronize).

```
# send edits from the local machine to the online version (the remote)
git push -u origin main
```

On the reverse, if we want to import new changes from a remote we use `git pull`.

```
# receive edits from the remote and integrate them into the loacl repository
git pull origin main
```

Finally, if we want to import an entire repository from an online source (and also download it's entire `git` history) we can use `git clone`. (In this case the repository will already know it's remote "origin" — that would be the url that we download from during the cloning).

```
# download an existing git repositroy (including it's entire version controlled history)
git clone https://github.com/<usr_name>/project_folder.git
```